探討地震對建築物破壞程度影響

第六組

一、研究動機與研究目的：

1. 主題更改：

這份報告的題目好像與上課時報告的不太一樣？沒錯，因為我們在實作「全球災難分布」地圖的時候，發現其經緯度座標是假的，座標對不上國家，地震也隨機分布(正常會聚集在地震帶)，違反「使用真實資料」的研究初衷，所以在此之後我們更換了另一個「全球災害統計資料」。

但是，我們在對此新資料做數據熱力圖分析後，發現其特徵間的相關性都很低(0.15 ~ 0.4)，沒辦法進行有效預測，所以我們直接換了一個主題。

但這筆資料是正確的，還是能預測全球災害種類發生機率。我們使用了 KNN 分類(KNeighborsClassifier)預測災害機率，並運用後端 API 與 docker 容器化，在虛擬主機上部署前端與後端，再多使用 Nginx 建立 swarm 分擔運算負擔，讓使用者能再地圖上點一個點就能快速計算其災害機率。但這不在「數據分析」的內容上，所以在此不再贅述。

2. 研究動機：

那為什麼最後選到這份數據？最近，1227 發生規模 7.0 的大地震，雖然震度不大，也沒有傷亡，但其恐怖還是讓我們心有餘悸。所以我們決定研究與地震相關的資料，這時我們在 Kaggle 上找到這份歷史地震資料：「Earthquake Magnitude, Damage and Impact」，內容經過官方認證，絕對真實，而其詳細內容在下方會統一說明。

3. 研究目的：

我們的研究目的是去預測當時如果有一棟房子，我們有其房子的物理結構、地理位置特徵，是否能預測成功其地震後的受損程度？使用除了目標變因「damage_grade」(受損程度)、與震後特徵的其他特徵，使用隨機森林模型(RF)去預測其受損程度。

二、資料介紹與資料處理：

原本在「Earthquake Magnitude, Damage and Impact」的資料庫中，資料內容極多，其內容量高達 1GB。但我們在簡單的觀察後，選擇了

「csv_household_earthquake_impact」這份資料，因為其內容才是可以分析的。

2015 年 4 月，尼泊爾甘達基省發生規模 7.8 地震。這場地震造成近 9,000 人喪生，數百萬人瞬間流離失所，經濟損失高達 100 億美元，約佔尼泊爾名目 GDP 的一半。而這份資料就是災後尼泊爾中央統計局提供。此資料除了有 30 個欄位，還有高達 76 萬筆資料，可以說明這份資料內容極其龐大。

1. 資料欄位簡短介紹

(1) 識別與地理位置

- building_id：每一棟建築物的識別碼。
- district_id：行政區代碼，層級最高的地理分區。
- vdcmun_id：村發展委員會或是市鎮代碼 。
- ward_id：鄰里代碼 ，最小的地理行政單位。

(2) 建築物理規格

- count_floors_pre_eq：地震前的樓層數。
- count_floors_post_eq：地震後剩餘的樓層數(震後數據，不採用)。
- age_building：建築物的屋齡。
- plinth_area_sq_ft：建築物的基座佔地面積(平方英尺)。
- height_ft_pre_eq：地震前的建築高度(英尺)。
- height_ft_post_eq：地震後的建築高度(震後數據，不採用)。

(3) 結構細節與環境

- land_surface_condition：地表狀況 。
- foundation_type：地基類型。
- roof_type：屋頂類型 。
- ground_floor_type：一樓地面類型 。
- other_floor_type：其他樓層的類型 。
- position：建築四周建築數量 。
- plan_configuration：建築平面形狀 。

(4) 上部結構材質

- has_superstructure_adobe_mud：是否使用土坯/泥磚。
- has_superstructure_mud_mortar_stone：是否使用泥漿砌石。
- has_superstructure_stone_flag：是否使用石板。
- has_superstructure_cement_mortar_stone：是否使用水泥砂漿砌石。
- has_superstructure_mud_mortar_brick：是否使用泥漿砌磚。
- has_superstructure_cement_mortar_brick：是否使用水泥砂漿砌磚。

- has_superstructure_timber：是否使用木材。
- has_superstructure_bamboo：是否使用竹材。
- has_superstructure_rc_non_engineered：是否使用非工程設計的鋼筋混凝土
- has_superstructure_rc_engineered：是否使用工程設計的鋼筋混凝土
- has_superstructure_other：是否使用其他材質。

(5) 損害評估與結果

- condition_post_eq：地震後的建築狀況描述 。
- **damage_grade：受損等級 (目標變數)**。由 1 到 5，輕微到嚴重。
- technical_solution_proposed：經評估後建議的解決方案。

2. 數據清洗：

(1) 資料預覽：

簡單的顯示每個欄位的資料型態，為之後的字串編碼做準備。

(2) 資料清洗：

使用 isnull()檢查是否有空缺，並計算每個欄位的空缺數。因為此資料筆數龐大，空缺值又僅有 26 筆，所以使用 dropna()直接刪除數據。

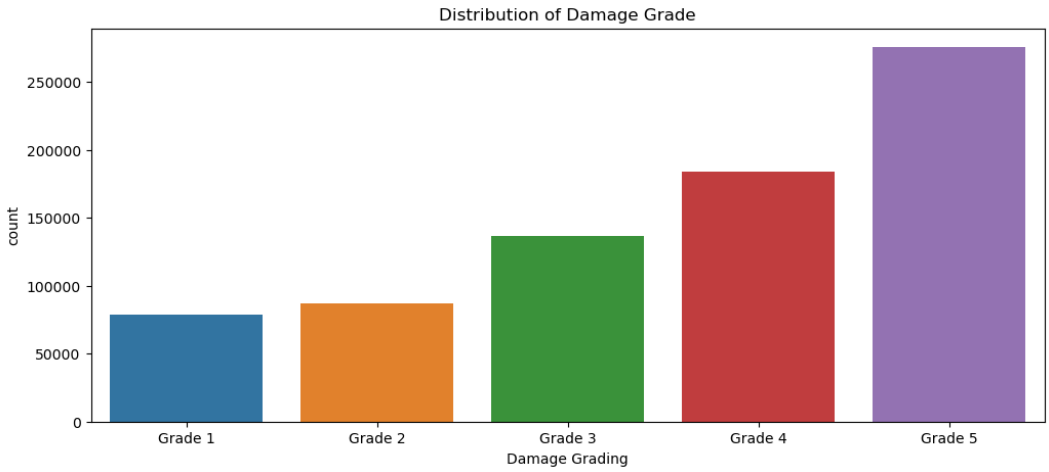
三、數據分析

1. 受損等級分佈

我們利用 sns.countplot() 函數針對建物受損等級（Damage Grade）進行樣本數量的統計與呈現。為了確保分析結果具有直觀的邏輯性，我們透過 order 參數將受損程度由輕微到嚴重依序排列，讓我們迅速了解建築不同損害程度的占比，以便於設計合適的圖表尺寸與標籤。而經過圖表，我們可以得知：

(1) 資料集的受損等級主要集中在 Grade 4 與 Grade 5。

(2) 這顯示該地區在地震中受災情況極為嚴重。



我們認為有下列幾種原因造成：

(1) 災害強度極大：這份數據是規模 7.8 的地震，且皆為在尼泊爾境內的數據，範圍小震度大，全毀的機率高合理。

(2) 建築耐震能力普遍不足： 該地區的建築可能多為老舊建物、土坏房或未符合現代耐震規範的構造，導致在災害發生時，較高的機率全毀或是嚴重受損。

2. 字串編碼與平滑處理

在特徵工程階段，我們首先使用 LabelEncoder 將原本屬於文字類別的資料轉換為數值格式，建立特徵與目標變數 X_encoded 以利後續運算。接著，對地理資訊、地基類型與屋頂構造等多個文字類別特徵，我們用了平滑目標編碼（Smooth Target Encoding）。這種方法是定義了一個 smooth_mean_encoding 函數，使用平滑參數 m = 5 當權，將該編碼化特徵的平均值與其特徵種類總數進行加權平均。

那為什麼要做字串編碼與平滑處理？：

(1) 地理風險量化：代表了該地區**連續**的「歷史受損嚴重程度」，而不只是單純沒有意義**不連續**的鄰里編號，能在之後的相關係數分析與模型分析使用。

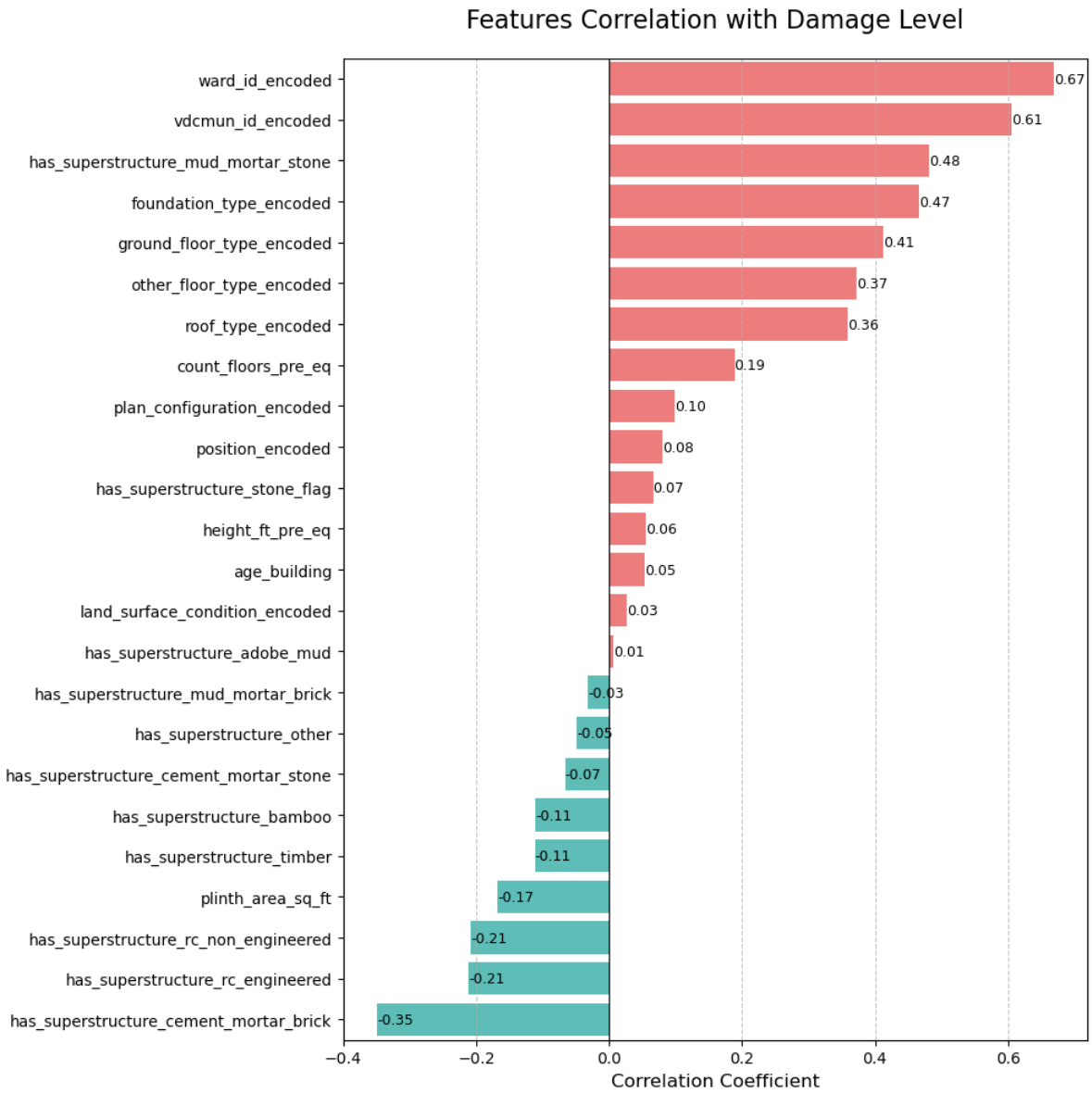
(2) 結構特質轉換：除了能讓數字化的特徵加入預測運算，也能使用這些物理特徵做一些特徵之間的整合，如脆弱性分析，它將不同的地基形式轉化為耐震分數，讓模型能直接比較不同結構的脆弱度。

但平滑編碼不是會有用到「受損等級」嗎？這樣測試的答案也混進去了吧？

雖然計算過程混入了震後數據，不過那是該地區**過去**的平均倒塌率，我們要**預測的是未來有地震時建築的受損程度**，並不是要穿越過去預測那次地震，所以平滑處理是視為該地區未來的「地理脆弱度指數」，而非地震後的直接震後數據。

3. 特徵相關性分析：

我們首先整合了編碼後的地理特徵、建築材料特徵以及建築基本物理屬性，並運用 corr() 函數計算各項特徵與受損等級之間的相關係數，藉此量化不同變數對建築損害程度的影響力。為了讓分析結果更易於解讀，我們將計算出的係數依數值大小進行排序，並透過橫向長條圖直觀地呈現結果。



根據上圖，我們可以知道：

(1) **地區的影響：**ward_id(鄰里)、vdcume_id(市鎮代碼)在平滑編碼後有最高的相關性，說明位置是最影響受損程度的。其原因可能包含該地區與震央的距離、當地的地質組成，以及不同地區在建築規範執行力上的差異。

(2) **脆弱材質的影響：**例如泥漿砌石、土坯等與受損等級呈現顯著的正相關。這證明了上面提到的泥漿砌石是最脆弱的結構。其原因也是因為傳統的石造建築搭配泥漿砂漿缺乏抗拉強度，在地震搖晃時石塊間極易滑脫導致整體崩塌。

(3) **穩定材質：**鋼筋混凝土、木材與受損等級呈負相關，是因為使用水泥砂漿或鋼筋混凝土的建物具有比較好的結構韌性，能夠吸收並消散地震能量，因此在數據上表現出明顯的保護作用。

下圖是我們加入一些建築的物理特徵，分別有：

(1) 每層樓建築體積(volume) = 建築面積(plinth_area_sq_ft) * 每層建築高度(height_ft_pre_eq)

(2) 建築細長比(slenderness) = $\frac{\text{每層建築高度}(height_ft_pre_eq)}{\sqrt{\text{建築面積}(plinth_area_sq_ft)+1}}$

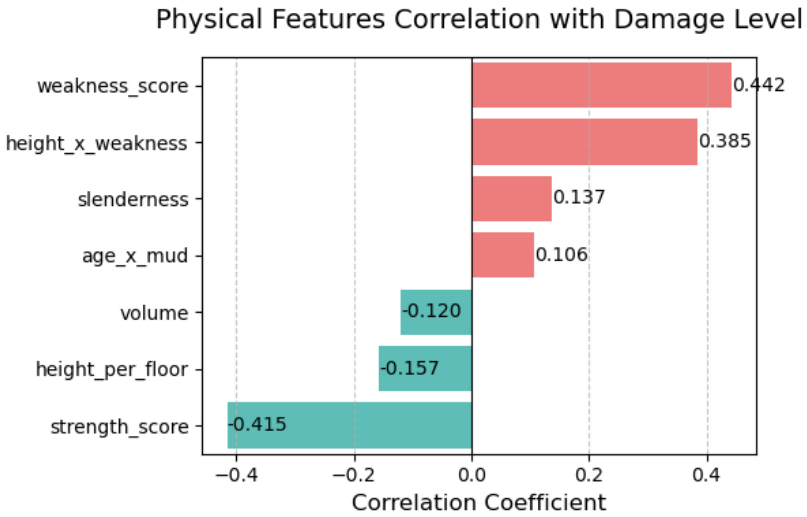
(3) 泥漿房屋齡(age_x_mud) = 屋齡(age_building) * 是否使用泥漿砌石(has_superstructure_mud_mortar_stone)

(4) 每層樓高度(height_per_floor) = 每層建築高度(height_ft_pre_eq) / 建築最高樓層數(count_floors_pre_eq)

(5) 脆弱性指數(weakness_score) = 使用脆弱材料數(包含：泥漿砌石、土坯/泥磚、石板)

(6) 使用脆弱材料的每層高度(height_x_weakness) = 每層建築高度(height_ft_pre_eq) * 脆弱性指數(weakness_score)

(7) 穩定性指數(strength_score) = 使用穩定材料數(包含：水泥砌磚、非工程設計的鋼筋混凝土、工程設計的鋼筋混凝土)



根據上圖，我們可以知道我們自己定義的物理特徵，與受損等級有符合定義的相關性，像脆弱性指數與穩定性指數分別都有正相關與負相關，接下來也會把這些物理特徵放入預測模型中。

四、模型預測結果：

1. 模型前處理：

在預測前處理階段，我們首先合併了受損等級，將原始的五類受損等級簡化為「低、中、高」三個層次。透過定義 merge_grades 函數，我們將受損最輕微的 Grade 1 定義為**不需處理**、Grade 2 與 3 合併為**需修復**、Grade 4 與 5 劃分為**重建**，這樣也符合現實中出現的「紅、橙、綠單」的分類方式，簡單的把建築分類成**是不適合居住**。而我們之所以不使用五種等級，是因為模型難以辨認 2 3 4 等級的差異，導致模型準確率卡在 0.57 上不去，而這種合併能有效降低類別間的雜訊，從而提升模型的預測穩定性。

接著，我們根據先前的分析結果篩選出關鍵特徵組成特徵集合，其中包括了經過平滑編碼的地理資訊與構造類型、建築物的基本物理指標，以及由我們自己定義的物理特徵，**總共 24 個特徵**。這套特徵組合不僅涵蓋了建築的原始數據，更透過物理特徵的交叉運算強化了數據的表達能力。最後，我們將處理後的標籤與特徵清單分別給 y 與 X，完成了從原始數據到結構化模型輸入的流程，為後續的分類預測奠定了基礎。

2. 模型訓練：

在模型訓練與評估階段，我們首先利用 train_test_split 函數將資料集劃分為訓練集與測試集，並特別設定 stratify = y 參數，讓三個受損等級在分割後仍能維持原始比

例。接著，我們定義了一個**隨機森林分類器（RandomForestClassifier）**，透過設定 100 棵決策樹以及 class_weight = 'balanced'，為了讓模型能自動平衡各類別的權重，避免因樣本不均導致預測偏向特定等級。還有我們設定 min_samples_leaf = 3，為了讓模型不被極端異常的資料影響，限制至少要選取 3 個以上的特徵。最後為了提升計算速度，我們多加了了 n_jobs = -1 以利用所有 CPU 核心進行並行運算。

至於選擇選用**隨機森林分類器**，第一他對**特徵很多的資料集效果很好**，也**擅長處理資料龐大**的資料集(SVM 就不適合)，也不用對資料做什麼預處理，因為訓練過程中會隨機選取特徵。還有能**不太會產生過擬合**，相較於決策樹(decision tree)容易鑽牛角尖，隨機森林透過 100 棵樹的投票，做出更客觀、更穩定的預測。最後是他能**提供特徵重要性**，因為這樣，我們才能想要增加什麼物理特徵增加準確率，還對最後的決策分析來說非常有價值。

當訓練完成後，計算整體的**準確率**來衡量模型的表現。而為了分析模型在三種受損層次上的細節能力，我們生成了詳細的分類報告，透過精確度、召回率以及 F1-score，檢核模型在不同分類任務中辨識能力。

3. 模型預測結果：

首先是預測準確率與 F1 分數。使用 feature_importances_函數看詳細的預測分數報告。

整體準確率：74.58%

評估指標	Level 1 (Low)	Level 2 (Mid)	Level 3 (High)
精確率	0.59	0.59	0.87
召回率	0.76	0.62	0.80
F1-Score	0.66	0.60	0.84

還算是不錯的結果，沒有過度擬合也沒欠擬合，但我們覺得還可以繼續往上衝，像是使用多模型的投票等等，但礙於訓練時間關係，一分半的訓練時間已足夠長了(對我們這個數據分析專題來說)。再看看我們的詳細結果：

(1)高受損等級預測極其精準：

對於「Level 3 需重建」的預測，精確率高達 87%，召回率也有 80%。使模型能非常可靠地識別出那些最危險、最需要重建的建築物。

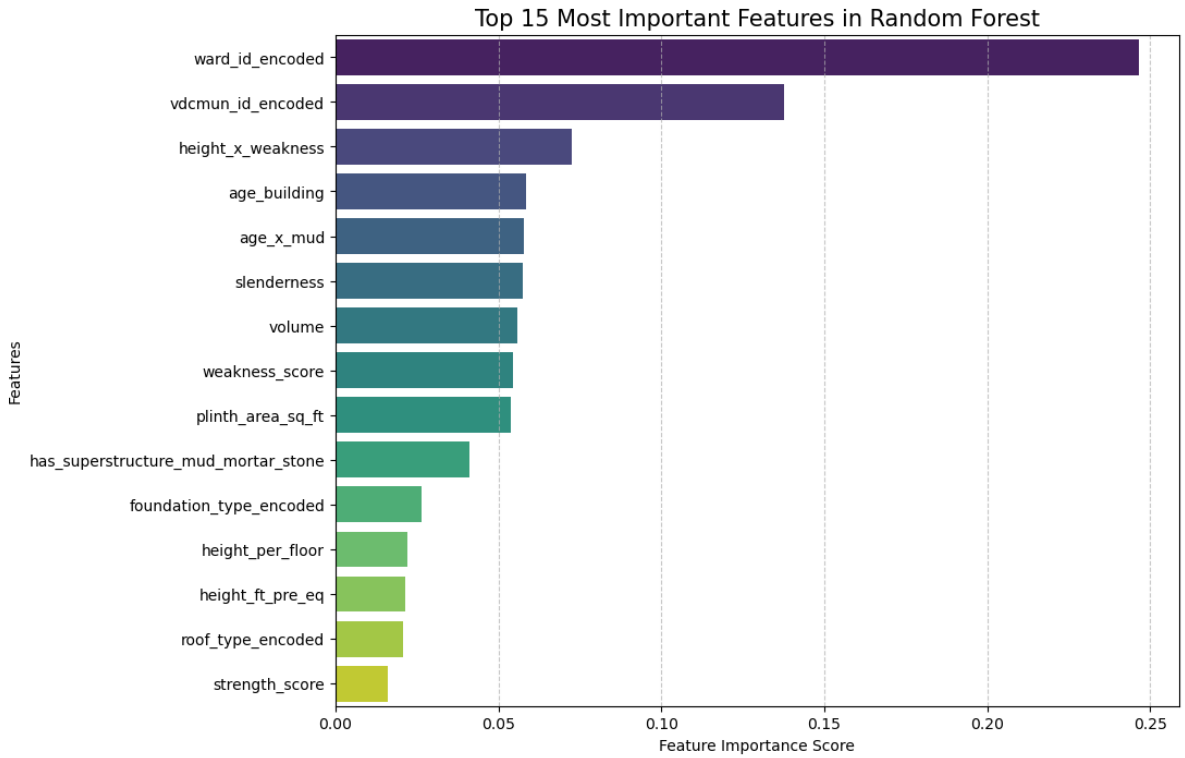
(2)低受損等級的安全性高：

對於「Level 1 無須處理」的召回率為 76%，代表大部分安全的建築都能被挑出來。

(3)中間模糊地帶難以分辨：

對於「Level 2 需修復」的 F1-Score 為最低 0.6，這代表了地震損害的連續性。中度受損與輕微或嚴重受損之間的界線在物理上較為模糊，這也是模型預測中最具挑戰的部分。

為了從眾多特徵中看到關鍵影響特徵，我們將特徵名稱與其對應的重要性得分整理成資料表，並依據得分高低進行降序排列，選取前十五個影響力最顯著的特徵進行排行。



前三名分別是：地區位置、使用脆弱材料的每層高度、屋齡，我們接下來會針對這三個做詳細的分析。

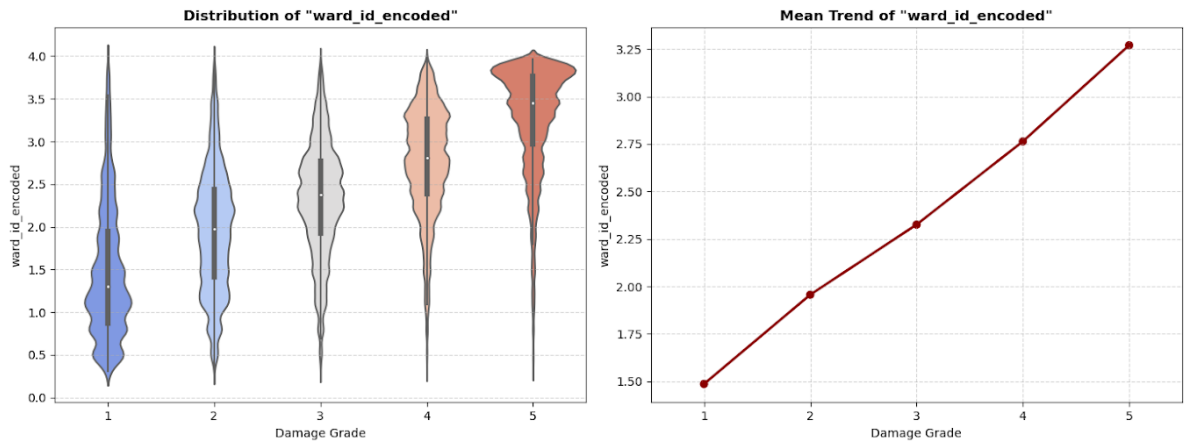
4. 重要特徵分析：

下圖前三重要特徵分析。

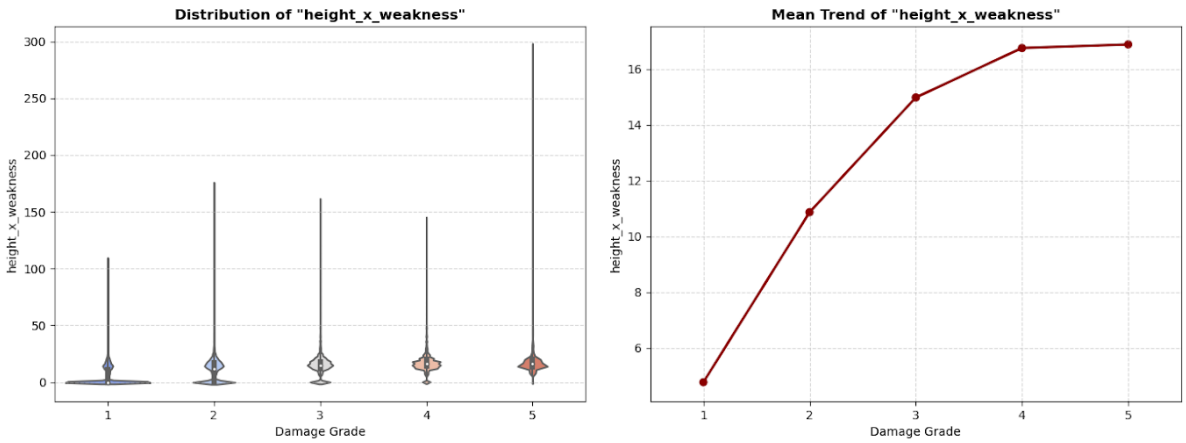
左圖：採用小提琴圖，觀察資料分佈的密度。若 Grade 5 的波峰明顯高於 Grade 1，代表該特徵能有效區分危險群體。

右圖：採用折線趨勢圖，觀察平均風險趨勢。

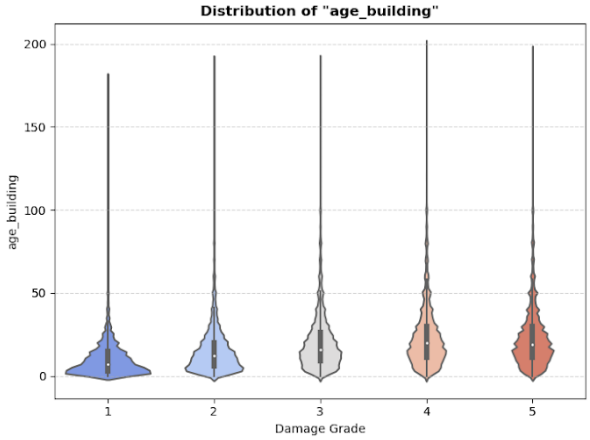
因排版關係，我們將此圖拆成三部分分開討論。

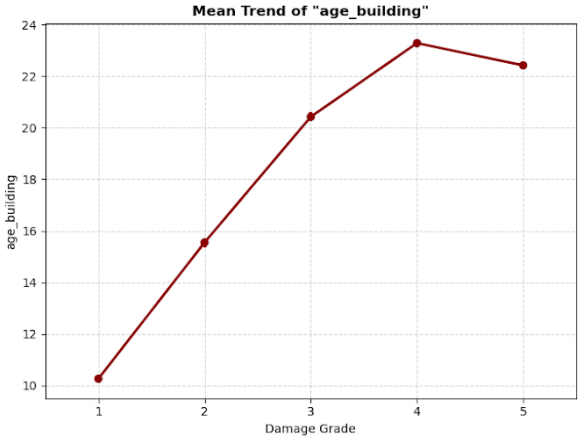


首先，地理位置是最重要的特徵，可能是因為**不同行政區與震央的距離**、當地的地表加速度、土壤的地質硬度差異導致，使得地理特徵成為模型判斷受損程度的最強指標。



而使用脆弱材料的每層高度物理特徵，說明**高層建築在地震時**，若**缺乏穩固結構支撐，其會增加受損機率**。最特別的是，在左上小提琴圖中，可以看到每個受損等級都基本有兩主要分布區域：上方的峰與下方的峰，上方的峰隨等級上升而變多，下方的峰隨等級上升反而變少，那是因為如果**沒有使用「脆弱性材料」**，其值會為零，正是**下方峰的族群**，隨**受損等級建築越來越少**也十分合理，反而如果**使用「脆弱性材料」**，其值會大於零，那就是上方峰的族群，會隨**受損等級愈來越多**。



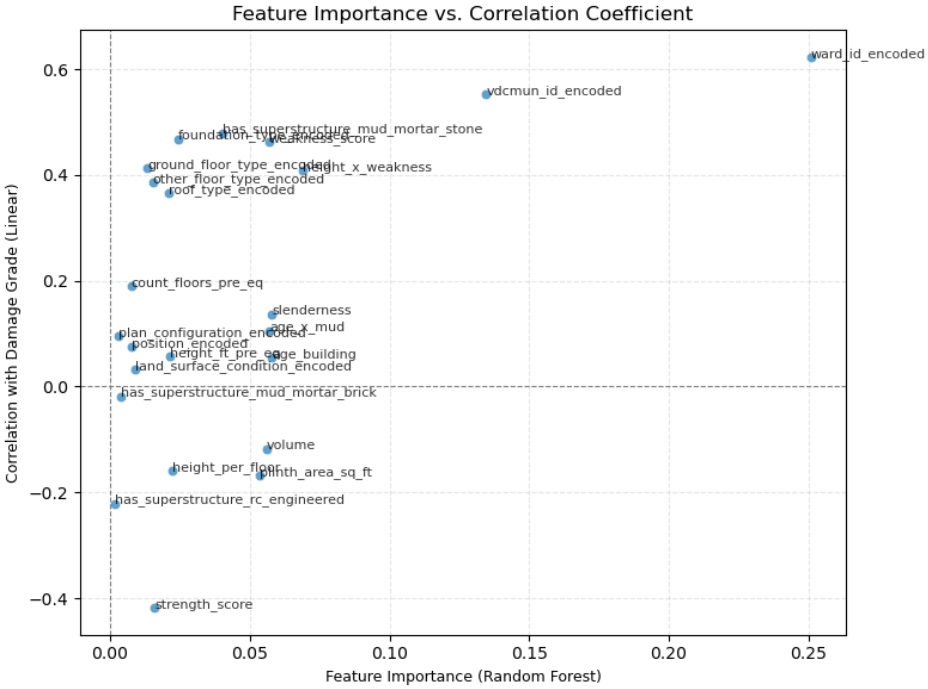


最後是屋齡，正常的建築會**隨時間推移**產生的**材料疲勞**以及舊有建築規範對耐震要求的不足，使得**老舊建築在應對強烈地震時更顯脆弱**。儘管在受損等級 5 其平均屋齡低於等級 4 ，但他的分布圖形幾乎差不多，推測是因為極端值影響平均，不影響整體趨勢，因為我們在預測時將等級 4 5 分成「需重建」等級。

5. 線性與非線性的比較：

在特徵評估的最後階段，我們比較隨機森林的特徵重要性與傳統的線性相關係數，繪製一張散佈圖。

首先，我們計算各特徵與目標變數之間的相關性，並與模型的重要性整合。接著，利用散佈圖將這兩種指標分別對應到 X 軸與 Y 軸，藉此觀察哪些特徵具備強大的線性解釋力。之後，我們還特別標註了重要性最高的前五名特徵，以及相關性最強的前三名特徵，並用虛線劃分出象限。這種分析方式能幫助我們有效識別出如地理編碼或特定的物理指標，是否在不同評價標準下都具有一致的影響力。



上圖，我們發現如屋齡這類特徵，雖然線性相關係數不一定最高，但在隨機森林中非常重要。這代表屋齡對受損的影響是非線性的。還有最下面的 strength_score，雖然他相關係數高，但他**資料數量很少**(多為 grade 1，數量最少)，**被隨機森林選中的機率低**，當然整體模型的重要性(也可以說是**影響力)較低**，也說明了重要度是「**辨別是否危險建築**」，對於安全建築反而不準。

五、結論：

我們透過資料科學方法分析尼泊爾地震數據，成功建立了具備 **74.58% 準確率**的受損預測模型，主要結果如下：

1. 我們利用隨機森林模型，針對高風險建築達到了 87% 的預測精確率與 80% 的召回率，能有效識別最危險的危樓，為災後資源配置提供可靠依據。
2. 透過特徵重要性分析，發現「地理位置」、「使用脆弱材料的每層高度」與「屋齡」是辨別**是否危險建築**三大核心要素。
3. 經由「特徵重要性 vs. 線性相關係數」的交叉比對，證實了如屋齡等特徵對倒塌風險具有**非線性影響**，這也是隨機森林模型優於傳統線性回歸的主因，還有說明這個模型的重要度是「**辨別是否危險建築**」。
4. 最後，我們前面自行定義的物理特徵在小提琴圖中出現了「雙峰」現象，成功將建築區分為「傳統脆弱結構」與「現代工程結構」，證明了特徵工程能有效強化模型的鑑別力。

六、AI 協作與 debug 紀錄：

我們在這次專題中，AI 的存在是最不可或缺的，他幫助我們高效率的執行我們天馬行空的想法，也減少我們眼乾乾盯著螢幕 debug 的時間。

1. AI 協作：

我們的 AI 協作是先給予 AI 一個方向，再看其展現的成果，如果覺得方向不合預期，就會再思考換另一種方向。像是我們決定使用平滑編碼時，決定平滑係數(m = 5)是 AI 提供意見，還有設定 min_samples_leaf = 3，也是使用 AI 生成 validation curve 幫我們選定，確保不會過擬合與欠擬合。

除此之外，選定模型也是透過 AI 協作，我們先是嘗試最常使用的 KNN 分類、線性回歸、決策樹(因為我們有很多 1/0 資料)，但結果都不如預期，最後是經過 AI 推薦使用隨機森林模型，非常適合我們多特徵的資料。

2. debug 紀錄：

我們在這次專題中碰碰撞撞的不停在 debug，像是剛開始的「全球災難分布」的假數據，也讓我們苦惱好久要不要改數據，之後更改數據後，也是遇到大麻煩就是不知道分析什麼(相關係數都很低)，最後才決定直接改主題。

改完主題後，debug 也不曾消失。像是在決定模型的階段，先嘗試線性回歸時，發現其特徵線性關係不明顯，分數大概只有 0.3。此後，我們也嘗試高次回歸模型，但是發現除了計算時間拉長外，還會產生過擬合的情況，因此，我們在特徵中加入平滑化，也加入模型的平滑化，但結果也不合預期。

在經過 AI 推薦之下，我們最後使用隨機森林模型，結果當然是在我們嘗試之中最好的，不過還卡在 0.52 的分數，最後加入了物理特徵，才成功將準確率推到 0.57，但仍不合預期，不過在與資料 technical_solution_proposed 欄位與受損等級的比較之下，決定把受損等級 5 種合併成 3 種，最終才到 0.75。